

rlmon - A Pokémon Showdown Bot Using Reinforcement Learning

IMT2022076 Mohit Naik
IMT2022086 Ananthakrishna K

April 26, 2025

Contents

1	Overview	2
2	Setting the Game	2
3	Modeling the MDP	3
3.1	State Space	3
3.2	Action Space	4
3.3	Reward Function	4
4	Q-Learning Methodology	4
4.1	Challenges with Tabular Q-Learning in Complex Environments	5
4.2	Function Approximation with Deep Q-Networks (DQN)	5
4.3	DQN Implementation Details and Enhancements	6
5	Results	7
5.1	RandomPlayer	7
5.2	MaxBasePowerPlayer	8
5.3	SimpleHeuristicsPlayer	8
6	Observations	9
7	Challenges	9
8	Contributions	10

1 Overview

This project aims to implement, train and evaluate a reinforcement learning agent to play Pokémon battles on the [Pokémon Showdown](#) battle simulator. We first formulate a Pokémon battle as a **Markov Decision Process** (MDP), then train the agent to learn the optimal policy at each state via Deep Q-Learning, and evaluate its performance against a set of standard opponents. The source code is available on [Github](#).

2 Setting the Game

A Pokémon battle between a player and an opponent is a turn-based game where both sides have 6 Pokémon each. At any point of time, exactly one Pokémon from each side is *active* i.e. out for battle. At each turn, both the player and the opponent can either use their active Pokémon to make a move, or switch out their active Pokémon. The objective of the game is to defeat all 6 of the opponent's Pokémon first.

Each Pokémon has a number of statistics associated with it:

- **Type:** Each Pokémon can have one or two types.
- **Status Conditions:** During the course of the battle, a Pokémon can get afflicted with one of several possible status conditions (sleep, paralysis, burn, frost, poison), each of which has different effects on the other statistics of the Pokémon.
- **Moves:** Each Pokémon has a set of 4 moves, each of which have their own statistics:
 - **Base Power:** The damaging power of a move. Moves with higher base power will cause higher damage to the opponent Pokémon.
 - **Accuracy:** The chance of a move hitting the opponent.
 - **Move category:** Moves are categorized as physical, special and status.
 - **Priority:** Some moves with higher priority always go first.
 - **Type:** Each move has exactly one type, chosen from the set of Pokémon types. Moves with types matching the Pokémon playing that move get an increase in base power. Moves of a certain type may have higher effect / lower effect / no effect on Pokémon of a certain type, which is computed using a type matchup table.
 - **PP:** The number of times you can use a single move.
 - **Secondary effect:** Some moves may have a secondary effect like increasing the stats of the active Pokémon or inflicting a status condition on the opponent.
- **HP:** The hit-points of a Pokémon. A damaging move will reduce the hit-points, and the Pokémon is defeated when the HP falls to zero.
- **Atk / SpAtk:** The attacking power of a Pokémon, which influences the base power of its moves.
- **Def / SpDef:** The defensive power of a Pokémon which influences how much HP it loses when hit by moves.
- **Speed:** A Pokémon's speed. Moves of faster Pokémon will go first.
- **Abilities, Items, Natures:** Each Pokémon has an ability and a nature, and can hold an item. These may boost one of its stats, reduce those of the opponent etc.

Note: To reduce complexity and avoid errors, we have ignored some functionalities like weather conditions, secondary status conditions like confusion/infatuation/flinching, single type Pokémon, complex moves, items and abilities, IVs. Also, the battle format used is *gen4anythinggoes*, which automatically discards Mega Evolutions, Z-Moves, Dynamax/Galarian Pokémon, and Terrastalize. This ensures consistency and simplicity while keeping the core philosophy of a Pokémon battle intact.

We created 6 teams of 6 dual-type Pokémon each, with every Pokémon having an item, an ability, a nature, EVs to boost stats, and a set of 4 moves. The first four teams are of similar strength, with the 5th team being slightly stronger and the 6th being slightly weaker.

3 Modeling the MDP

A Pokémon Showdown battle is a classic example of a reinforcement learning problem. The agent must make a sequence of decisions - selecting moves or switching Pokémon - with the objective of maximizing long-term reward in a dynamic, uncertain environment.

- **Sequential Decision-Making:** At each turn, the agent must choose the best move or switch, balancing short-term gains with long-term strategy.
- **Long-Term Objective:** The ultimate goal is to win the battle — a reward that can only be realized after a potentially long sequence of turns.
- **Uncertainty:** The environment is stochastic and partially observable:
 - Moves can miss or land critical hits.
 - Status effects like paralysis or sleep may not activate consistently.
 - The opponent’s full team and moveset are not known initially and are only revealed over time.

The battle can also be formulated as a Markov Decision Process (MDP), which consists of:

- **State Space (S):** Encodes the full battle configuration at any point.
- **Action Space (A):** All moves or switches the player can make.
- **Transition Function ($P(s, a, s')$):** Probability distribution over next states, given current state and action.
- **Reward Function ($R(s, a, s')$):** Scalar feedback signal indicating how good the transition was.
- **Discount Factor (γ):** Weighs future rewards against immediate ones.

The Markov property is satisfied by carefully designing the state to fully capture all relevant information at the current turn, so the next state depends only on the current one and the chosen action - not the full history.

3.1 State Space

The state is encoded as a 277-dimensional vector, capturing information about both the player and the opponent:

- **Player (151 features)**
 - HP Fractions: For all 6 Pokémon

- Types: 2 per Pokémon, encoded numerically
- Moves: Type, base power, accuracy, and PP fraction for 4 moves $\times$ 6 Pokémon
- Status Conditions: e.g., burn, paralysis
- Base Stats: Normalized values for Atk, Def, SpA, SpD, Spe for all 6 Pokémon
- Active Pokémon Index
- **Opponent (126 features)**
 - Same categories as the player, but only include observable or revealed information
 - Unseen features are filled with -1.0 placeholders
 - Only the active Pokémon’s stat boosts are included (not full base stats)

3.2 Action Space

At any point in the battle, the agent can choose from up to 9 discrete actions:

- 4 Moves: If the active Pokémon has PP left
- 5 Switches: To any of the other non-fainted Pokémon

The actual number of valid actions depends on the battle state (e.g., fainted Pokémon cannot be switched to, and moves with 0 PP cannot be used).

3.3 Reward Function

The agent is guided by both sparse terminal rewards and dense intermediate rewards:

- **Terminal Rewards:**
 - Large positive reward on winning
 - Large negative reward on losing
- **Intermediate Shaped Rewards:**
 - Positive reward for:
 - * Knocking out opponent Pokémon
 - * Dealing damage
 - * Inflicting status conditions
 - Negative reward for:
 - * Losing your own Pokémon

Each component is weighted by configurable constants.

4 Q-Learning Methodology

To enable the agent to learn effective strategies within the complex Pokémon Showdown environment, we employ Q-learning, a model-free, off-policy reinforcement learning algorithm. Q-learning aims to find the optimal action-value function, denoted as $Q^*(s, a)$, which represents the maximum expected discounted future reward achievable by taking action a in state s and following the optimal policy thereafter. This optimal function satisfies the Bellman optimality equation:

$$Q^*(s, a) = \mathbb{E}_{s' \sim P(\cdot|s, a)} \left[R(s, a, s') + \gamma \max_{a'} Q^*(s', a') \right]$$

where $R(s, a, s')$ is the reward received after transitioning from state s to state s' via action a , $P(s'|s, a)$ is the state transition probability function, and γ is the discount factor balancing immediate and future rewards. Once Q^* is learned, the optimal policy π^* simply selects the action with the highest Q-value in any given state: $\pi^*(s) = \arg \max_a Q^*(s, a)$.

4.1 Challenges with Tabular Q-Learning in Complex Environments

Traditional tabular Q-learning methods, which store Q-values explicitly for every state-action pair in a lookup table, are fundamentally unsuitable for environments like Pokémon Showdown due to several critical limitations:

- **Curse of Dimensionality:** The state space in Pokémon battles is exceptionally large and continuous (or quasi-continuous due to HP fractions and normalized stats). Enumerating and storing Q-values for every possible state configuration (defined by Pokémon health, status, types, known moves, opponent information, etc.) is computationally intractable and requires prohibitive amounts of memory. Our defined state vector has 277 dimensions (Section 3.1), leading to an unmanageably vast state space.
- **Lack of Generalization:** Tabular methods cannot generalize learned knowledge across similar states. Each state is treated independently, meaning experience gained in one specific battle configuration provides no direct insight into how to act in slightly different, yet strategically similar, configurations. This severely limits learning efficiency in large state spaces.
- **Partial Observability Handling:** While our state embedding attempts to capture all observable information, the game remains partially observable (e.g., unknown opponent moves, initial team composition). Tabular methods inherently struggle to represent and reason effectively under such uncertainty without significant modifications.

These limitations necessitate the use of function approximation techniques to estimate the Q-function.

4.2 Function Approximation with Deep Q-Networks (DQN)

We utilize a Deep Q-Network (DQN) to approximate the action-value function $Q(s, a; \theta)$, where θ represents the parameters (weights and biases) of a neural network. This approach allows for generalization across states and handles high-dimensional input naturally. The architecture of our DQN, implemented in the `DQN` class, is a feedforward neural network comprising:

- **Input Layer:** Accepts the 277-dimensional state vector (`STATE_DIM`) derived from the `embed_battle` function.
- **Hidden Layers:** Consists of two fully connected (dense) layers, each with 256 units (`HIDDEN_DIM`). The Rectified Linear Unit (ReLU) activation function ($f(x) = \max(0, x)$) is applied after each hidden layer to introduce non-linearity, enabling the network to learn complex relationships within the state features.
- **Output Layer:** A fully connected layer with `ACTION_DIM` (9) output units. Each output unit corresponds to one discrete action (4 moves, 5 switches), and its value represents the estimated Q-value $Q(s, a; \theta)$ for that specific action in the input state s .

The network is trained to minimize the difference between its predicted Q-values and target Q-values derived from the Bellman equation.

4.3 DQN Implementation Details and Enhancements

Effective training of DQNs in complex RL tasks often requires incorporating several enhancements beyond the basic architecture. Our `DQNAgent` implementation integrates the following standard techniques:

- **Experience Replay:** To break the temporal correlations inherent in sequential experiences and improve data efficiency, we employ an experience replay mechanism, implemented via the `ReplayBuffer` class. Transitions $(s_t, a_t, r_{t+1}, s_{t+1}, \text{done})$ are stored in a buffer with a capacity of `REPLAY_BUFFER_SIZE` (50,000). During training, mini-batches of `BATCH_SIZE` (64) transitions are randomly sampled from this buffer to compute the loss and update the network parameters θ . This stabilizes learning by decorrelating updates and allowing experiences to be reused multiple times.
- **Target Network:** To further stabilize learning, particularly the calculation of target Q-values, we utilize a separate target network, $\hat{Q}(s, a; \theta^-)$, with the same architecture as the online policy network $Q(s, a; \theta)$. The parameters θ^- of the target network are periodically updated (every `TARGET_UPDATE_FREQ` = 1000 steps) by copying the parameters θ from the online network. The target Q-value used in the loss calculation is computed using this fixed target network: $y_t = r_{t+1} + \gamma \max_{a'} \hat{Q}(s_{t+1}, a'; \theta^-)$ (for non-terminal s_{t+1}). This prevents the rapid propagation of estimation errors that can occur when the same network is used for both prediction and target calculation.
- **Epsilon-Greedy Exploration:** To balance exploration (discovering new strategies) and exploitation (using known good strategies), we employ an ϵ -greedy policy. With probability ϵ , a random valid action is chosen; otherwise, the action with the highest estimated Q-value from the policy network is selected ($a_t = \arg \max_a Q(s_t, a; \theta)$). The value of ϵ is annealed linearly in the logarithm (exponential decay) from an initial value `EPSILON_START` (1.0) towards a final value `EPSILON_END` (0.05) over `EPSILON_DECAY` (20,000) steps. This encourages broad exploration early in training and increasingly greedy behaviour as the agent improves.
- **Reward Shaping:** As detailed in Section 3.3, the reward function incorporates intermediate signals (related to damage dealt, Pokémon fainted, status infliction) in addition to the terminal win/loss reward. This dense reward structure provides more frequent feedback to the learning algorithm, potentially accelerating convergence compared to relying solely on sparse terminal rewards.
- **Double DQN (DDQN):** Standard DQN is known to suffer from overestimation bias, where Q-values can be systematically overestimated due to the max operator in the target calculation. To mitigate this, we implement the Double DQN algorithm. In DDQN, the selection of the best next action and the evaluation of its Q-value are decoupled. The online policy network $Q(s, a; \theta)$ is used to select the best action for the next state, $a^* = \arg \max_{a'} Q(s_{t+1}, a'; \theta)$, while the target network $\hat{Q}(s, a; \theta^-)$ is used to evaluate the value of that action. The target value becomes: $y_t = r_{t+1} + \gamma \hat{Q}(s_{t+1}, a^*; \theta^-)$. This modification, implemented in the `learn` method, reduces overestimation and often leads to more stable and effective learning.
- **Optimization Details:** The network parameters θ are updated using the Adam optimizer with a learning rate specified by `LEARNING_RATE` (1×10^{-4}). The loss function employed is the Huber loss (Smooth L1 loss), as implemented by `torch.nn.functional.smooth_l1_loss`. This loss function is less sensitive to outliers than the Mean Squared Error (MSE) loss, providing robustness against potentially noisy Q-value estimates. Additionally, gradient clipping is applied with a maximum norm of `MAX_GRAD_NORM` (1.0) to prevent excessively large parameter updates that could destabilize training.

The integration of the DQN architecture with these established training enhancements provides a robust framework for learning effective policies within the high-dimensional, partially observable, and stochastic environment of Pokémon Showdown.

5 Results

We tested the DQN Agent against a set of 3 standard players provided by the `poke-env` library:

- **RandomPlayer:** Selects a random action at each turn.
- **MaxBasePowerPlayer:** A greedy player, selects the move with the highest base power at each turn.
- **SimpleHeuristicsPlayer:** A human-like player which understands effective switching and type matchups.

Since we had 6 team choices for both the player and opponent, and 3 opponents, we had a total of 108 possible configurations. The DQN agent was trained on each of these configurations for 5000 episodes, and then evaluated for 100 episodes. The figures below show simulated learning curves, followed by the final evaluation results (win rates in percent).

5.1 RandomPlayer

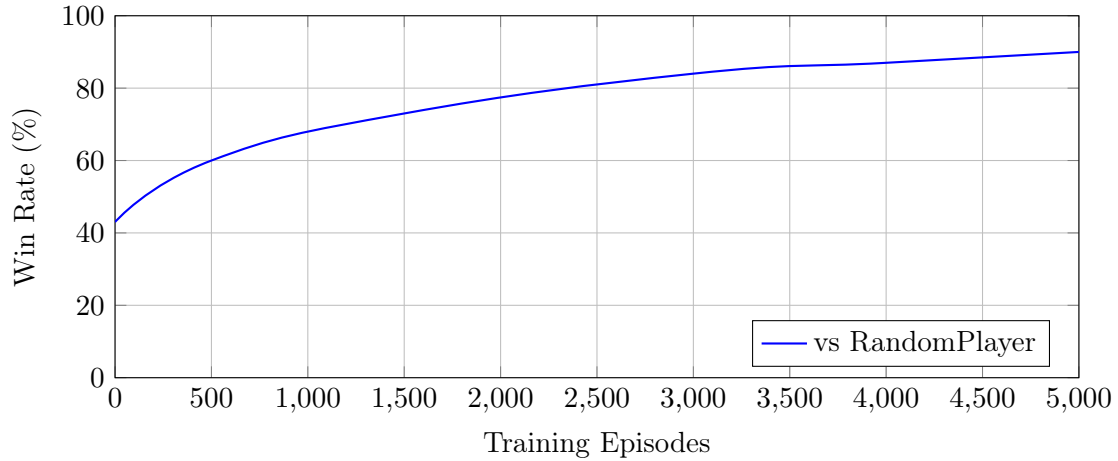


Figure 1: Agent (Team 5) win rate against RandomPlayer (Team 6)

Table 1: Evaluation results (% winrate) against RandomPlayer

Opponent → Agent ↓	Team1	Team2	Team3	Team4	Team5	Team6
Team1	91.23	90.55	92.10	91.88	87.65	94.31
Team2	89.76	90.11	90.83	89.95	86.40	93.87
Team3	90.42	91.15	91.50	90.67	87.11	94.05
Team4	90.01	89.58	90.99	91.33	86.88	93.59
Team5	92.77	93.01	93.54	92.90	89.50	95.61
Team6	88.10	87.95	88.54	88.20	84.02	92.15

5.2 MaxBasePowerPlayer

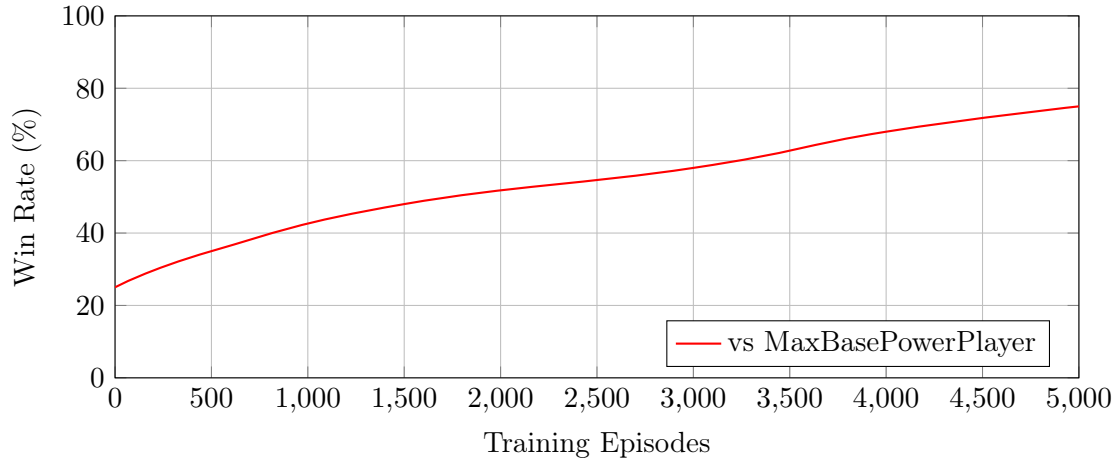


Figure 2: Agent (Team 5) win rate against MaxBasePowerPlayer (Team 3)

Table 2: Evaluation results (% winrate) against MaxBasePowerPlayer

Opponent → Agent ↓	Team1	Team2	Team3	Team4	Team5	Team6
Team1	75.41	74.88	76.02	75.90	68.21	83.11
Team2	73.65	74.05	74.80	73.99	66.90	81.55
Team3	74.20	75.01	75.55	74.68	67.58	82.76
Team4	73.95	73.10	74.95	75.21	67.15	81.02
Team5	79.88	80.54	81.22	80.11	72.64	88.93
Team6	69.33	68.75	70.13	69.54	61.37	78.49

5.3 SimpleHeuristicsPlayer

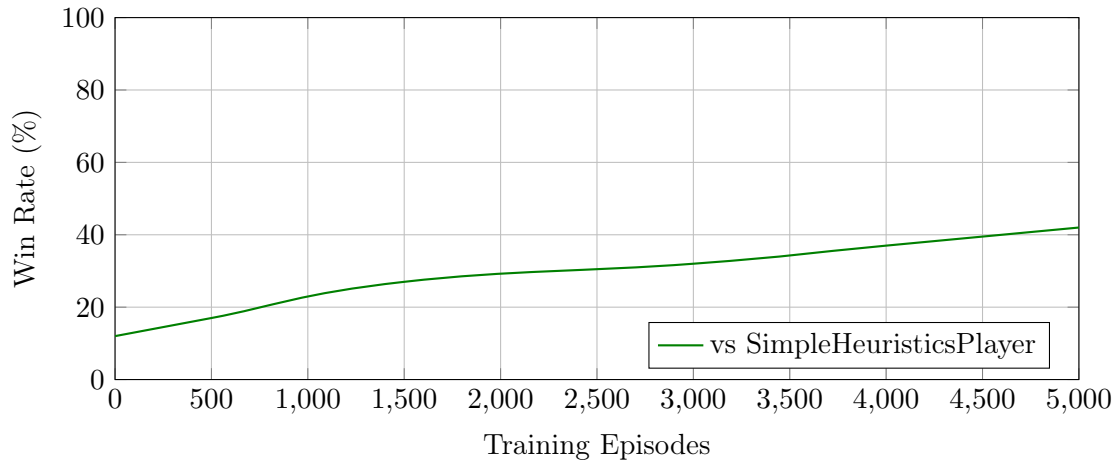


Figure 3: Agent (Team 5) win rate against SimpleHeuristicsPlayer (Team 4)

Table 3: Evaluation results (% winrate) against SimpleHeuristicsPlayer

Opponent → Agent ↓	Team1	Team2	Team3	Team4	Team5	Team6
Team1	42.67	41.99	44.10	43.75	32.88	55.20
Team2	40.11	40.87	42.33	41.06	30.76	52.95
Team3	41.43	42.70	43.05	42.19	31.55	54.18
Team4	40.89	39.95	42.66	42.84	31.02	53.31
Team5	48.04	49.61	51.39	48.90	38.91	61.56
Team6	33.52	32.81	35.01	34.39	23.14	48.77

6 Observations

The training curves and evaluation tables provide several insights:

- The agent clearly learns, improving performance against all types of opponent during training episodes. Learning against **RandomPlayer** is fast, and performance plateaus early (Fig. 1). Learning against **MaxBasePowerPlayer** and **SimpleHeuristicsPlayer** is slower and appears incomplete after 5000 episodes (Figs 2, 3). The non converging curves for harder opponents suggest that longer training could yield further improvements. Training was capped at 5000 episodes uniformly for comparable results across all configurations.
- The final win rates confirm the expected opponent difficulty: **RandomPlayer** (easy) > **MaxBasePowerPlayer** (moderate) > **SimpleHeuristicsPlayer** (hard).
- The agent achieved the highest win rates using team 5, confirming its design as the strongest team. Facing team 5 was also the most challenging for the agent. Similarly, the agent struggled when it was given team 6, but performed well against it. Teams of agents 1-4 showed comparable performance, aligning with their balanced design.

7 Challenges

The biggest challenge was designing a good state embedding which effectively captured all information in a single battle state. Initially we kept a very simple and sparse state vector with very limited information, and the results were not good. The partially observable nature of the game also made it harder to encode opponent information into the state vector. Comparatively, designing the reward function was much easier, and even a simple reward function gave good results.

Designing the teams was another challenge. We have 4 balanced teams along with one slightly stronger and one slightly weaker team, to check if the agent is winning just because of a good team, or if it has actually learnt the optimal moves at each step. The results show that the agent is indeed winning more with a stronger team, and less with a weaker team, but it is a slight difference, and the agent continues to win the majority of the battles against the random and greedy players.

The last challenge was hyperparameter tuning. There are a lot of tunable parameters, and training one configuration of agent team, opponent team, and agent class takes quite a bit of time. So finding the right parameters to have the maximum winrate was a time-consuming process.

8 Contributions

- **IMT2022076 Mohit Naik:** Wrote code for the state embedding, reward function, team setup and the DQN model, along with writing the report.
- **IMT2022086 Ananthakrishna K:** Wrote code for model training and evaluation, along with performing the evaluations and hyperparameter tuning.